



ELSEVIER

Computer Physics Communications 152 (2003) 208–226

Computer Physics
Communications

www.elsevier.com/locate/cpc

GMIC++: Grouping method in C++: an efficient method to solve large number of Master equations[☆]

A.M. Ovcharenko^a, S.I. Golubov^a, C.H. Woo^{a,*}, Hanchen Huang^b

^a Department of Mechanical Engineering, Hong Kong Polytechnic University, Hong Kong

^b Department of Mechanical, Aerospace & Nuclear Engineering, Rensselaer Polytechnic Institute, Troy, NY 12180, USA

Received 1 February 2002; received in revised form 30 July 2002

Abstract

In typical nucleation, growth and coarsening problems in the study of defect/adatom accumulation in crystalline solids or surfaces, a large number of Master equations are involved to describe the evolution process. As examples, defect clusters nucleate and grow from point defects in solids when subjected to particle irradiation, and atoms depositing on a substrate form clusters leading to film growth. To efficiently solve the large number of master equations, the grouping method was used, which we have coded into a standard C++ program, taking full advantage of the object-oriented programming style supported in the C++ language. Because of the generic nature of this code, it may be of interest to the modeling nucleation and growth processes. As an example to demonstrate the application of this computer code, the Ostwald ripening process of vacancy clustering during aging in metal nickel is calculated.

© 2002 Elsevier Science B.V. All rights reserved.

PACS: 61.72.Cc; 61.72.Ji; 68.43.Jk

Keywords: Grouping method; Master equation; Nucleation; Coarsening; Clusters

PROGRAM SUMMARY

Computer for which the program is designed and others on which it has been tested: PC

Title of program: GMIC++

Operating systems under which the program has been tested: Windows NT/2000/XP, Cygwin (UNIX environment for Windows™), Linux Red Hat v8.0.

Catalogue identifier: ADRK

Program Summary URL: <http://cpc.cs.qub.ac.uk/summaries/ADRK>

Programming language used: C++

Program obtainable from: CPC Program Library, Queen's University of Belfast, N. Ireland

Memory required to execute with typical data: 500 Kb

[☆] This program can be downloaded from the CPC Program Library under catalogue identifier: <http://cpc.cs.qub.ac.uk/summaries/ADRK>

^{*} Corresponding author.

E-mail address: chung.woo@polyu.edu.hk (C.H. Woo).

No. of bytes in distributed program, including test data, etc.: 42 378

Distribution format: gzip file

CPC Program Library subprograms used: none

Nature of the physical problem

A common approach to describe defect/adatom accumulation processes is by using a system of master equations. For a good description, the number of equations in the system is generally very large. As a result, their numerical solution is very time consuming. This problem is less serious at the early stage. However, as the process continues, the number of master equations increases and the numerical difficulty becomes severe.

Method of solution

To physically track the details of the accumulation process with available computational resources, Kiritani [1] first proposed the group method. Accordingly, the system of master equations is divided into groups, keeping the first moments unchanged. Realizing

the inadequacy of the original grouping method, Golubov et al. [2] proposed to preserve also the second moment, which has been shown to provide an accurate description of the defect/adatom accumulation processes, as originally described by a large number of master equations.

Restrictions on the complexity of the problem

The method is valid for general defect/adatom accumulation processes, which are describable by a set of master equations.

Typical running time

It depends on energetics and the speed of the processes involved. As an example, it takes 30 s of a Pentium III processor at 1 GHz clock frequency to track the vacancy clustering in nickel during one hour aging at 550°C, solving a system of up to 300 master equations.

References

- [1] M. Kiritani, J. Phys. Soc. Japan 37 (1974) 1532.
- [2] S.I. Golubov, A.M. Ovcharenko, A.V. Barashev, B.N. Singh, Philos. Mag. A 81 (2001) 643.

LONG WRITE-UP

1. Introduction

Nucleation, growth and coarsening are generic processes in the study of defect/adatom accumulation in crystalline solids or surfaces. A set of kinetic equations—commonly known as Master equations—describes a distribution of entities as a function of time [1]; the entities would be defect clusters of various sizes [2–9]. Establishing the set of equations is straightforward. However, numerical solution is anything but easy. As the number of species of entities goes up, say to a million, the numerical solution of a million master equations over a large number of time steps (say a billion) would be prohibitively intensive in computation, if not impossible within foreseeable future.

This generic problem manifests itself in the form of evolution of point defects/adatom clusters, generated due to annealing, irradiation, or various kinds of deposition techniques, as a function of time. To numerically solve this evolution problem, Kiritani proposed a grouping method [3], in which all the clusters in one group are assumed to have one average size, and this size obeys one Master equation. To conserve the total number of clusters and the total number of defects in the clusters, as defined in the original Master equations, Kiritani introduced effective reaction constants. This approach has been applied to solving problems for both aging and irradiation (see e.g. [3, 8,9]).

Although the physical robustness of this approach has been challenged back in 1974 [10], a full analysis has been carried out only recently [11]. It has been shown that this method is inadequate since it leads to the distortion of the validity of the physical process described by the original kinetic equation. Such a distortion occurs due to the fundamental assumption [3] that the cluster densities (and their increments) are equal within one group, thus breaking the required conservation laws [11]. Golubov et al. [11] derived the correct effective reaction constants that represent the physical process more accurately. The numerical results of the improved grouping method agree well with the analytical solution of model problems, in contrast to the substantial disagreement between the original method and the analytical solutions.

Due to the generic nature of the defect/adatom accumulation processes, a standard code solving a set of Master equations describing evolution of point defects (PD) size distribution function (SDF) using the new group method, would be useful to the study of many classes of problems.

In this paper, we present a computer code, GMIC++, for the solution a large system of Master equations. In Section 2, we briefly describe the improved grouping method. We present the computer program in Section 3. The input and output data are explained in Section 4, and sample data and the corresponding test run are presented in Section 5.

2. The grouping method and model description

2.1. Kinetic equation (Master equation)

The kinetic equation used in [11] to describe the evolution of the size distribution function, $f(x, t)$, has the form of a continuity equation in the discrete space of cluster size (x is the number of defects in a cluster)

$$\frac{\partial f(x, t)}{\partial t} = J(x-1, t) - J(x, t), \quad x \geq 2, \quad (1)$$

where $J(x, t)$ is the flux of clusters in space of cluster size. For example, in the simple case of Ostwald ripening process $J(x, t)$ is given by

$$J(x, t) = P(x, t)f(x, t) - Q(x+1, t)f(x+1, t). \quad (2)$$

In this case $P(x, t)$ is the rate of a point-defects absorption by a cluster of size x , and $Q(x, t)$ is the rate of emission/dissolution from the cluster (see [11] for an example). It follows from Eq. (1) that the total number of clusters, $N(t) = \sum_{x=2}^{\infty} f(x, t)$, and the total number of point-defects/adatom in the clusters, $S(t) = \sum_{x=2}^{\infty} xf(x, t)$, are described by the following equations

$$\frac{dN(t)}{dt} = J(1, t), \quad (3a)$$

$$\frac{dS(t)}{dt} = J(1, t) + \sum_{x=1}^{\infty} J(x, t). \quad (3b)$$

Eq. (3), which represents the conservation laws for the process under consideration, is the key ingredient of the improved grouping method [11]. In contrast to the Kiritani method, this new development ensures these conservation laws.

2.2. Grouping method

Before considering the grouping algorithm let us define groups. Assume that clusters are grouped with the group width $\Delta x_i = x_i - x_{i-1}$, where the subscript i indicates the number of a group and the i th group contains clusters of the sizes from $x = x_{i-1} + 1$ to x_i . It can be easily shown that the mean size of the i th group,

$$\langle x \rangle_i = \frac{1}{\Delta x_i} \sum_{k=1}^{\Delta x_i} (x_{i-1} + k)$$

obeys the equation $x_i = \langle x \rangle_i + (\Delta x_i - 1)/2$. Note that in a limiting case of $\Delta x_i = 1$ the mean size of i th group is $\langle x \rangle_i = x_i$ and $x_{i-1} = x_i - 1$ i.e., ungrouped clusters can be considered as a limiting case of the grouped ones. In the case when the ungrouped clusters are used for small sizes, which is a common practice (see e.g. [3,8]), the mean size of the i th group is simply equal to the number of the group, $\langle x \rangle_i = x_i = i$.

As mentioned above the main objective of the grouping method is to reduce the number of evolution equations. In the Kiritani method [3] a single equation for an average cluster concentration is used instead of Δx_i equations describing the clusters of each particular size. However it has been shown [11] that such an approximation for size distribution function within a group leads to the misrepresentation of the physical process defined by Eq. (1). In order to establish a better physical approximation for the size distribution function, the exchange of point defects between different groups has to be analyzed in a more rigorous way.

Let us consider the i th group containing Δx_i clusters of different sizes. Summing Eq. (1) over the i th group, one can obtain the equation for the total number of clusters accumulated in the group, $N_i = \sum_{k=1}^{\Delta x_i} f(x_{i-1} + k)$

$$\frac{\partial N_i}{\partial t} = J(x_{i-1}) - J(x_i). \quad (4)$$

Multiplying Eq. (1) by x and summing over the group, one can obtain the equation for the total number of point defects accumulated in clusters of this group, $S_i = \sum_{k=1}^{\Delta x_i} (x_{i-1} + k) f(x_{i-1} + k)$

$$\frac{\partial S_i}{\partial t} = (x_{i-1} + 1)J(x_{i-1}) + \sum_{k=1}^{\Delta x_i - 1} J(x_{i-1} + k) - x_i J(x_i). \quad (5)$$

Eqs. (4) and (5), representing the conservation laws for each group of clusters, give a clue as to why the Kiritani method fails and how a proper approximation may be developed. Indeed it has been shown [11] that the approximation of the size distribution function within a group used in [3] does not allow both conservation laws to be satisfied and has to be replaced by a more realistic approach. As it has been shown in [11] the simplest way to do it is to approximate the size distribution function by a linear function within a group:

$$f(x) = L_0^i + L_1^i(x - \langle x \rangle_i). \quad (6)$$

Substituting Eq. (6) to Eqs. (4) and (5), one can find [11]:

$$\frac{\partial L_0^i}{\partial t} = \frac{1}{\Delta x_i} [J(x_{i-1}) - J(x_i)], \quad (7a)$$

$$\frac{\partial L_1^i}{\partial t} = -\frac{\Delta x_i - 1}{2\Delta x_i \sigma_i^2} [J(x_{i-1}) + 2\langle J \rangle_i^* - J(x_i)], \quad (7b)$$

where $\langle J \rangle_i^*$ is the mean flux inside the i th group,

$$\langle J \rangle_i^* = \frac{1}{\Delta x_i - 1} \left[\sum_{k=1}^{\Delta x_i - 1} J(x_{i-1} + k) \right], \quad (8)$$

and σ_i^2 is the dispersion of cluster size in the group,

$$\sigma_i^2 = \langle x^2 \rangle_i - \langle x \rangle_i^2 = \frac{1}{\Delta x_i} \left[\sum_{i=x_{i-1}+1}^{x_i} i^2 - \frac{1}{\Delta x_i} \left(\sum_{i=x_{i-1}+1}^{x_i} i \right)^2 \right]. \quad (9)$$

Note that the mean flux $\langle J \rangle_i^*$ is meaningless at $\Delta x_i = 1$, i.e. it has to be taken to be equal 0.

Eq. (7) describe the evolution of the size distribution function with the group approximation (in [11] the multiplier $-1/\Delta x_i$ in the right-hand side of Eqs. (38) was lost). It has already been pointed out in [11] that within the grouping method there is no problem in describing the size distribution function in the region between the ungrouped and grouped clusters since in the case $\Delta x_i = 1$ Eqs. (7) are transformed to Eq. (1) ($f(x_i) = L_0^i$, $L_1^i = 0$).

Eqs. (6)–(9) can be applied to calculate the size distribution function of many types of physical phenomena described by the continuity equation (1). As it has already been pointed out above, this includes the evolution of the distribution function at all stages (nucleation, growth and coarsening) of different cluster types (vacancy

and interstitial clusters, secondary phase precipitates, adatoms, etc.) under aging, deposition, and irradiation in the discrete space of the cluster sizes. However, the reaction constants have to be given to realize a computation. For clarity and simplicity, we choose the specific case of the clustering process of vacancies as a prototype. The computer code, for different sets of reaction constants, applies to the different nucleation and growth processes.

2.3. Model description

In the following, we describe the model using the specific example of vacancy clustering during aging. Generalization of the application to other cases is straightforward. In this case, the reaction rate $P(x, t)$ and the dissociation rate $Q(x, t)$ in Eq. (2) take the form [11]

$$P(x, t) = wx^{1/3}D_v C_v(t), \quad x \geq 1, \quad (10a)$$

$$Q(x, t) = wx^{1/3}D_v \exp\left(-\frac{E_v^b(x)}{k_B T}\right), \quad x \geq 2, \quad (10b)$$

where $w = (48\pi^2/\Omega^2)^{1/3}$ (Ω is the atomic volume), $C_v(t)$ and D_v are the concentration and diffusion coefficient of vacancies, respectively; $E_v^b(x)$ is the binding energy of a vacancy with a cluster of size x .

The vacancy concentration, C_v , is given by:

$$\frac{dC_v(t)}{dt} = -k^2 D_v (C_v(t) - C_{v0}) - J(1, t) - \sum_{x=1}^{\infty} J(x, t), \quad (11)$$

where k^2 is the total sink strength of all extended defects in a crystal (excluding that of vacancy clusters); C_{v0} is the thermal equilibrium concentration of vacancies.

The binding energy is given by the capillary model:

$$E_v^b(x) = E_v^f - 2\gamma\Omega/r, \quad (12)$$

where E_v^f is the vacancy formation energy, r is the radius of void and it is related to x by $x\Omega = 4\pi r^3/3$. In the general case, one also could use other forms of the binding energy, as, for instance, in the description of the evolution of copper clusters in iron [5,7].

The initial and boundary conditions are taken to have the following form

$$f(x, t=0) = C_q \delta(x-1), \quad C_v(t=0) = C_q, \quad (13a)$$

$$f(x=1, t) = C_v(t), \quad f(x=\infty, t) = 0, \quad (13b)$$

where C_q is the quenched-in vacancy concentration. Eqs. (2), (6)–(13) have been used in this paper to design the program GMIC++. In this regard, it must be noted that the present code has been designed to serve as a base, on which a code for the solution of kinetic problems of specific interest, such as aging, deposition, and irradiation, can be readily developed.

3. Program structure

3.1. Programming language

It is desirable that the code has a flexible design that can be easily modified by users. The object-oriented programming (OOP) language is suited for this purpose. The program is developed in C++ programming language that supports OOP and is the most effective one in comparison (see e.g. [12]).

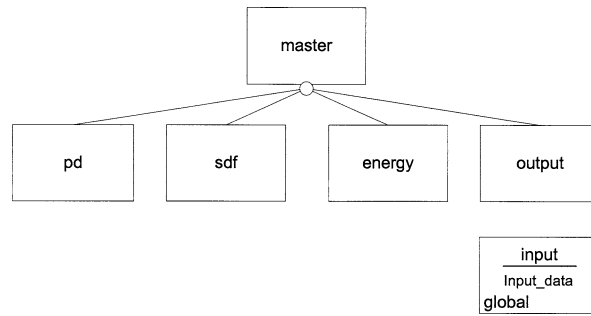


Fig. 1. Top-level class diagram comprises three levels of abstraction.

3.2. Top-level class diagram (transparent scheme of the system)

The program comprises six essential class categories expressed in terms of C++ *namespaces*, which serve to partition the logical model of the program. These namespaces contain logically related classes, structures and free subprograms. In addition, the program includes a few necessary classes and structures outside the namespaces.

The class diagram suggested by Booch in [13] is used to present this program. Fig. 1 presents top-level class diagram containing only class categories, which represent the high-level design of our program. This diagram shows layers of the system, where a layer denotes the collection of class categories (i.e. *namespaces*) at the same level of abstraction. Thus, layers represent grouping of class categories, just as class categories represent clusters of classes. A common use of layers is to insulate higher layers from lower layer details. Based on this approach, our system is divided into three layers. The highest layer contains *master* namespace, medium one contains *pd*, *sdf*, *energy* and *output* namespaces and the lowest layer contains *input* namespace. Note that the key word *global* given in the input box is used to indicate that *Input_data* structure is used by all class categories in the system.

3.3. Classes description

What follows is a detailed description of the classes included in the listed namespaces.

3.3.1. Master namespace

The *master* namespace collects classes providing services for general management over all the computed data and its safe storage. It belongs to the highest level of abstraction and contains four classes and one subprogram listed below.

```

namespace master {
//Master data managers
    class Master_mgr;
    struct IMaster_agnt;
    class Master_agnt;

//Master data
    class Master_data;

// Group generator
    Vector Get_groups(size_t);
}
  
```

Master data managers

The *Master_mngr* class does not have a base class. It is the top-level class of the system, which provides highest-level functionality controlling over almost all the components and operations of the system. Its public interface is:

Method (public)	Description
<i>Master_mngr()</i>	Constructs a <i>Master_mngr</i> object.
<i>Before_run()</i>	Checks whether it is the first run, which is the case when the binary file for continuation is absent. In the case it deletes previously written files, otherwise restores information saved in binary file to continue interrupted computation.
<i>Run()</i>	Runs general calculation cycle.

The *Master_mngr* includes a pointer of *IMaster_agnt* type that can refer to instances of any classes related to *IMaster_agnt* as a common superclass. In this program *Master_agnt* is the single class that inherits its public interface from *IMaster_agnt*. This interface is given below:

Method (public)	Description
<i>Restore_data()</i>	Restore the state of the <i>Master_data</i> object saved before in binary file (virtual).
<i>Set_data()</i>	Sets the current state of an object (virtual).

Moreover *Master_agnt* contains *Master_data* object and also includes instances of the following classes: *pd::PD_mngr*, *sdf::SDF_mngr* and *output::Output_mngr*.

Master data

The *Master_data* class does not have a base class. It serves for a general storage of currently computed data providing a complete set of various methods controlling them. It also includes both read and write methods to support object's ability to save and restore its own state through a binary file. Almost all class data are defined as static that allows them to be shared between any instances of this class. Its public interface is given in Table 1.

Set_time_step() is a method, which allows to set up the changeable time step, $\Delta t(t)$, for numerical integration of Eqs. (7). The method used in the program is described in Appendix A.

Resize() is an efficient method used to enlarge the number of the groups during a calculation. This procedure is based on the assumption that SDF evolves from the smallest sizes to the larger ones. This method increases the number of groups by 5 extra groups if the concentration of clusters in the terminal group becomes greater than a certain limit determined by input (see "Limiting clusters number density for the terminal group" in Section 4). At the same time it increases the size of all arrays used in the computation, thus providing necessary synchronization of array sizes.

Group generator

Get_groups() method is the only free subprogram of the *master* namespace providing generation of the group widths Δx_i (see Appendix B).

In Sections 3.3.2–3.3.5 the medium layer of proposed abstraction and public interfaces of classes included in the appropriate namespaces are described.

3.3.2. The *pd* namespace

The *pd* namespace unites classes that performs calculation of point defect concentration. It includes five classes and one subprogram listed in the following declaration:

Table 1

Method (public)	Description
<i>Master_data()</i>	Constructs a <i>Master_data</i> object.
<i>Save_copies()</i>	Saves copies of data (virtual).
<i>Write()</i>	Writes binary data (virtual).
<i>Read()</i>	Reads binary data (virtual).
<i>Set_time()</i>	Sets current aging time (2 overloads).
<i>Set_time_step()</i>	Sets time step (2 overloads).
<i>Set_data()</i>	Sets a current state of an object (3 overloads, virtual).
<i>Get_time()</i>	Returns value of current aging time (static).
<i>Get_time_step()</i>	Returns value of time step (static).
<i>Get_time_step_init()</i>	Returns value of initial time step (static).
<i>Get_time_step_max()</i>	Returns value of maximum time step (static).
<i>Get_cv()</i>	Returns value of vacancy concentration (static).
<i>Get_x_size()</i>	Returns current size of arrays (static).
<i>Get_a()</i>	Returns array of L_1^i coefficients (static).
<i>Get_c()</i>	Returns array of L_0^i coefficients (static).
<i>Get_txt_file_counter()</i>	Returns value of a counter controlling text file data records (static).
<i>Get_n()</i>	Returns array of the total number of clusters in each group (static).
<i>Get_s()</i>	Returns array of the total number of vacancies accumulated in each group (static).
<i>Get_qvs()</i>	Returns the total number of vacancies emitted from voids (static).
<i>Get_pvs()</i>	Returns the total number of vacancies captured by voids (static).
<i>Resize()</i>	Resizes arrays.
<i>Is_less()</i>	Checks if current aging time is larger then terminal time for calculation.

```

namespace pd {
//PD data manager
    struct IPD_mngr;
    class PD_mngr;

//PD data
    struct IPD_data;
    struct IVac_data;
    class Vac_data;

//Method for calculation of point defect concentrations
    double e_mtd(double, double, double, double);
}

```

PD data manager

The *PD_mngr* class is derived from pure virtual base class *IPD_mngr*. *PD_mngr* object manages calculations involving the mobile defect concentrations using the following public interface:

Method (public)	Description
<i>PD_mngr()</i>	Constructs a <i>PD_mngr</i> object.
<i>Set_data()</i>	Sets a current state of the object (virtual).
<i>Get_cv()</i>	Returns value of vacancy concentration.

PD data

The *PD_mgr* has a *IVac_data* type pointer that refers to *Vac_data* object. The *Vac_data* class inherits its public interface from the *IVac_data*, which in turn inherits from the *IPD_data*. Thus the *Vac_data* class public interface is:

Method (public)	Description
<i>Vac_data()</i>	Constructs a <i>Vac_data</i> object.
<i>Set_data()</i>	Sets a current state of the object (virtual).
<i>Get_vcr()</i>	Returns value of vacancy loss rate (virtual).
<i>Get_vbr()</i>	Returns value of vacancy generation rate (virtual).
<i>Get_cv()</i>	Returns value of vacancy concentration (virtual).

The instance of the *Vac_data* class is a part of *PD_mgr* class, providing *PD_mgr* object with necessary data and methods to perform calculations for vacancy concentration.

Method for calculation of point defect concentrations

Free subprogram *e_mtd()* offers a method for solution of Eq. (11) for point defect concentration.

3.3.3. The *sdf* namespace

The *sdf* namespace contains classes to calculate the evolution of the size distribution function. It includes four classes and two subprograms presented in the following declaration:

```
namespace sdf {
//SDF data manager
  struct ISDF_mgr;
  class SDF_mgr;

//SDF data
  struct ISDF_data;
  class SDF_data;

//Free subprograms used for numerical integration
  Vector Predict (const Vector&, const Vector&, double);
  Vector Correct (const Vector&, const Vector&, const Vector&, double);
}
```

SDF data manager

The *SDF_mgr* class is derived from pure virtual base class *ISDF_mgr*. This class is developed to manage *SDF_data* object and perform calculation of L_0^i and L_1^i coefficients. Its public interface is:

Method (public)	Description
<i>SDF_mgr()</i>	Constructs an <i>SDF_mgr</i> object.
<i>Set_data()</i>	Sets current state of the object (virtual).
<i>Get_qvs()</i>	Returns the total number of vacancies emitted from voids.
<i>Get_pvs()</i>	Returns the total number of vacancies captured by voids.
<i>Get_a()</i>	Returns an array of L_1^i coefficients.
<i>Get_c()</i>	Returns an array of L_0^i coefficients.

SDF data

The *SDF_mngr* has an *ISDF_data* type pointer that refers to the instance of *SDF_data* class inheriting its interface from *ISDF_data*. The *SDF_data* public interface is:

Method (public)	Description
<i>SDF_data()</i>	Constructs an <i>SDF_data</i> object.
<i>Set_data()</i>	Sets a current state of the object (2 overloads) (virtual).
<i>Get_jr()</i>	Returns array of right boundary fluxes ($J(x_i)$ in Eqs. (7)) (virtual).
<i>Get_jl()</i>	Returns array of left boundary fluxes ($J(x_{i-1})$ in Eqs. (7)) (virtual).
<i>Get_js()</i>	Returns array of mean fluxes ($\langle J \rangle_i^*$ in Eqs. (7)) (virtual).
<i>Get_v_emit()</i>	Returns the total number of vacancies emitted from voids (virtual).
<i>Get_x_capt()</i>	Returns the total number of vacancies captured by voids (virtual).

The instance of the *SDF_data* class is a part of *SDF_mngr* class, providing *SDF_mngr* object with necessary data to perform calculations for L_0^i and L_1^i coefficients.

The interface of *SDF_data* class includes *Get_jr()*, *Get_jl()* and *Get_js()* methods that return $J(x)$, $J(x_{i-1})$ and $\langle J \rangle_i^*$ fluxes, respectively, which are necessary to compute L_0^i and L_1^i coefficients by *SDF_mngr* class. Thus the calculation of Eqs. (7) is separated from the calculation of the fluxes, which can have a different form for different types of clusters. Such separation provides a simple way to replace the flux $J(x)$ used (see Eqs. (10)–(12)) by an appropriate one.

Free subprograms used for numerical integration

To integrate Eqs. (7) the *sdf* namespace contains two functions realizing so-called Modified Euler method [14]. This scheme, consisting one correction for predicted function value at every time step only, is used for the sake of simplicity.

3.3.4. The *energy* namespace

The *energy* namespace unites classes, which performs calculation the binding energy E_v^b . It includes three classes listed in the declaration below:

```
namespace energy {
//Energy data manager
    class Energy_mngr;

//Energy data
    struct IEnergy_data;
    class Energy_data;
}
```

Energy data manager

The *Energy_mngr* class does not have a base class. It is introduced to provide managing over binding energy data. The class public interface is:

Method (public)	Description
<i>Energy_mngr()</i>	Constructs an <i>Energy_mngr</i> object.
<i>Set_data()</i>	Sets current state of the object.
<i>Get_ed_ptr()</i>	Returns pointer to <i>IEnergy_data</i> interface.
<i>Check()</i>	Write the object to check file.

Energy data

The *Energy_mngr* class contains a pointer of *IEnergy_data* type, which may refer to the object of *Energy_data* class, inheriting interface from *IEnergy_data*. The *Energy_data* class public interface is:

Method (public)	Description
<i>Energy_data()</i>	Constructs an <i>Energy_data</i> object.
<i>Set_data()</i>	Sets current state of the object (virtual).
<i>Get_x()</i>	Returns array of group coordinates, x_i (virtual).
<i>Get_ev()</i>	Returns array of binding energy (2 overloads, virtual).
<i>Get_eev()</i>	Returns array of exponent of binding energy (virtual).

The *Energy_data* class represents the formulation of the binding energy, in the form of Eq. (12). The program assumes that an access to the methods of *Energy_data* class is performed via the pointer to *IEnergy_data* returned by the *Get_ed_ptr()* method of the *Energy_mngr* class. This provides a convenient way for different formulations of binding energy to be incorporated in the program using only the pointer to *IEnergy_data* interface.

3.3.5. The *output* namespace

The *output* namespace unites classes designed to manage output data. It includes nine classes in the following declaration:

```
namespace output {
// Output data manager
    struct IOutput_mngr;
    class Output_mngr;

//Output data
    class Output_sdfx;
    class Output_sdfz;
    class Output_scrn;
    class Output_pd;
    class Output_sums;
    class Output_rm
    class Output_xm
};
```

Output data manager

The *Output_mngr* class is derived from pure virtual base class *IOutput_mngr*. This class writes all necessary output data associated with different classes specified in the namespace declaration. Its public interface is as follows:

Method (public)	Description
<i>Output_mgr()</i>	Constructs an <i>Output_mgr</i> object.
<i>Set_data()</i>	Sets current state of the object (virtual).
<i>Write_scrn()</i>	Writes data to screen (virtual).
<i>Write_bin_file()</i>	Writes data to the binary file (virtual).
<i>Write_txt_file()</i>	Writes data to the text files (virtual).
<i>Get_scrn_counter()</i>	Returns current value of screen counter.
<i>Get_bin_file_counter()</i>	Returns current value of binary file counter.
<i>Get_txt_file_counter()</i>	Returns current value of text file counter.

Output data

The *Output_sdfx* and *Output_sdfz* classes contain SDF data, namely, functions $f(\langle x \rangle_i)$ and $f(\langle r \rangle_i)$ presented in so-called x and r spaces, respectively, where $\langle x \rangle_i$ and $\langle r \rangle_i$ are the mean number of defects in a cluster and the mean radius of the cluster in i th group, respectively.

The *Output_sums* class calculates the total number of clusters $N = \sum_{i=2}^{i_{\max}} L_0^i \Delta x_i$ and the total number of vacancies accumulated in clusters $S = \sum_{i=2}^{i_{\max}} (L_0^i \langle x \rangle_i + L_1^i \sigma_1^2) \Delta x_i$, where the subscript i indicates the number of a group.

The *Output_xm* and *Output_rm* classes calculate mean cluster size, $\langle x \rangle$ and $\langle r \rangle$, respectively, where $\langle x \rangle = S/N$ and $\langle r \rangle = (3\Omega \langle x \rangle / (4\pi))^{1/3}$.

The *Output_pd* class records vacancy concentrations.

The *Output_scrn* class performs screen print of the following parameters: aging temperature T , current aging time t , time step Δt , maximum time step $\Delta t_{\max}$, vacancy concentration C_v , diffusion coefficient D_v , vacancy flux $D_v C_v$, the total number of clusters N , the total number of vacancies accumulated in clusters S , mean cluster size $\langle x \rangle$ and $\langle r \rangle$.

In the following Sections 3.3.6 and 3.3.7 the lowest layer of proposed abstraction and a few classes that is not covered by namespaces, are described.

3.3.6. The *input* namespace

The lowest layer of abstraction consists of the *input* namespace that includes *Input_loader* class and *Input_data* structure.

```
namespace input {
    class Input_loader;
    struct Input_data;
};
```

The *Input_loader* class is dedicated to load input data from file to the *Input_data* structure. Public interface of *Input_loader* class is:

Method (public)	Description
<i>Input_loader()</i>	Constructs an <i>Input_loader</i> object.
<i>Load()</i>	Loads input data from file to <i>Input_data</i> structure.
<i>Check()</i>	Checks whether loading is correct.

3.3.7. Additional classes

In this section we list the rest number of essential classes that are not covered by the namespaces considered above. There are at least four classes and structures can deserve one's attention: *Vector*, *Group_data*, *Form* and

Const_data. The *Vector* is a class that represents an abstraction of single-dimensional array of floating-point values and implements all the necessary vector operations used in the program. This class is based on a standard *valarray* template, optimized to speed-up vector operations when performing numerical computations. To design vector operations we use a technique that allows highly optimized implementation suggested in [15].

The *Group_data* is a structure that contains data required for various computations with groups.

The *Form* is a class that simplifies the output of a floating-point number in a predefined format [15].

The *Const_data* is a structure containing all the necessary basic constants.

3.4. Entry point of the program

In this section we complete the description of the program and present a view of the entry point *main()* routine that appears very simple.

```
int main() {
    input::Input_loader il;
    il.Load();
    master::Master_mngr mm;
    mm.Before_run();
    mm.Run();
    return 0;
}
```

As seen, it just creates two objects of *input::Input_loader* class and *master::Master_mngr* class. After that it invokes the appropriate methods to load data from the input file and then run computations.

4. Input and output data

4.1. Input data

A single input data file */i_data.txt/* is required. Parameters demanded for GMIC++ reading from standard input are presented in Table 2.

4.2. Output data

In the output the following data are printed:

- time dependency of vacancy concentration (**o_vac.dat**);
- time dependency total number of clusters and total number of vacancies accumulated in clusters (**o_sum.dat**);
- time dependency of mean cluster radius (**o_rmv.dat**);
- size distribution function calculated in various size spaces (**o_sdf.dat**).

5. Test run description

The test data are the files of time dependence of vacancy concentration, $C(t)$ and the integral parameters describing the cluster evolution, N , S , $\langle x \rangle$, $\langle r \rangle$, and the SDF for the terminal aging time produced from the calculations of SDF for void during aging in nickel-like metal at 823 K. The concentration of the quenched-in vacancies and other material parameters are given in Table 3. The terminal time of aging is chosen to be equal 10^4 seconds to make the test-case results to be short.

Table 2

Type	Name	Default	Meaning
double	t	823	Aging temperature. Units: K
double	om	1.205e−29	Atomic volume. Units: m ³
double	ga	1000	Surface energy. Units: mJ/m ²
double	evf	1.77	Vacancy formation energy. Units: eV
double	evm	1.1	Vacancy migration energy. Units: eV
double	cv0	1.00	Pre-exponent coefficient for equilibrium concentration of vacancies
double	dv0	1.0e−06	Pre-exponent coefficient for diffusion coefficient. Units: m ² /sec
double	cq	1.0e−07	Initial vacancy concentration. Units: atom ^{−1}
double	kv2	0	Sink strength. Units: 1/m ²
double	time	1.0e+04	Terminal time for calculation. Units: sec
size_t	x_size	5	Initial number of groups (initial size of arrays)
size_t	x_sngl	10	Number of groups with the group widths equal to 1($\Delta x = 1$)
double	dr	2.0e−11	Group width in <i>r</i> -space. Units: m
double	cmax	1.0e−20	Limiting clusters number density of the terminal group for the resize procedure (Section 3.3.1). Units: atom ^{−1}
unsigned	wrScrn	100000	Number of cycles for subsequent screen output
unsigned	wrBinf	200000	Number of cycles for subsequent binary data conservation
unsigned	wrTxtf	1	Number of text data records during change in time of an order of magnitude
double	wrFrom	1.0e−00	First record time to write text files. Units: sec

Table 3

Temperature	823 K
Atomic volume, Ω	$1.205 \times 10^{-29} \text{ m}^3$
Vacancy diffusion coefficient, D_v	$10^{-6} \exp(-1.1 \text{ eV}/k_B T) \text{ m}^2 \text{ s}^{-1}$
Equilibrium vacancy concentration, C_{v0}	$\frac{1}{\Omega} \times \exp(-1.77 \text{ eV}/k_B T) \text{ m}^{-3}$
Surface energy, γ	1000 mJ/m ²
Concentration of quenched-in vacancies, C_q	$10^{-7} \text{ atom}^{-1}$
Sink strength, k^2	0.0
Time step growth parameter, A	0.5
Time step limitation parameter, B	1.7

To illustrate the test run results the SDF for the terminal aging time together with the results of the calculations performed to higher doses are given in Figs. 2 and 3. In Fig. 2 the SDF normalized to the total number of voids, i.e. a ratio $f(r, t)/N(t)$, is presented for aging times in a range from 10^3 to 10^7 . In Fig. 3 the function $f(r, t)$ is presented in a form, which allows comparing the calculated results with well-known Lifshitz–Slezov–Wagner model (LSW) (see e.g. [16]) describing the latest stages of the void coarsening (see Appendix C). As can be seen from Fig. 3 the calculated results are in a good agreement with LSW model that indicates an accuracy of the program described in this paper.

The test run results, which used default values for the input data are given at the end of this paper.

Appendix A

A choice of the time step, Δt , to produce results of sufficient accuracy with manageable CPU time, in the calculation of SDF evolution is always a challenging problem. In the program the time step is given by

$$\Delta t = \begin{cases} \frac{A}{D_v N(t)^{2/3}}, & \Delta t \leq \Delta t_{\max}, \\ \Delta t_{\max}, & \Delta t > \Delta t_{\max}, \end{cases} \quad (\text{A.1})$$

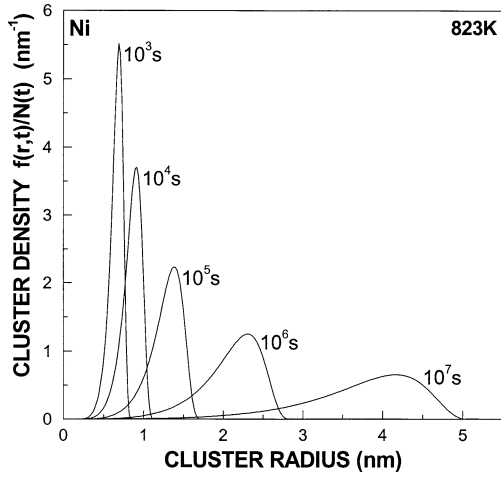


Fig. 2.

Fig. 2. Normalized SDF in r -space for different aging times.

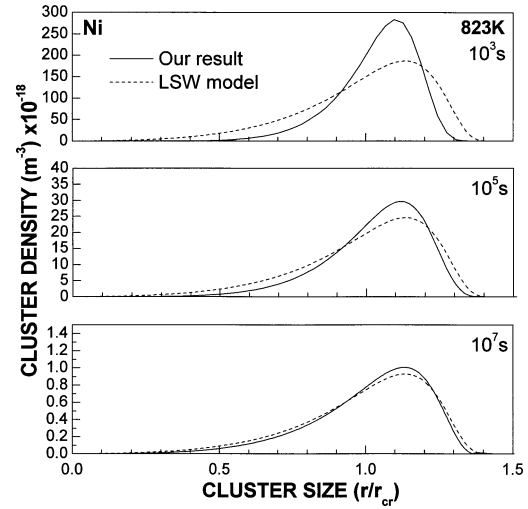


Fig. 3.

Fig. 3. Calculated SDF in the relative size space, $u = r/r_{cr}$ for different aging times. Dashed lines correspond to the LSW model (see Eq. (16)).

where $\Delta t_{\max} = B/Q_v(2)$, $N(t)$ is the void number density, A and B are constants (see Table 3). In Eq. (A.1) the time step increase with increase of aging time ($N(t)$ is decreased at coarsening regime) in a way, which allows to maintain a ratio of the mean square displacement of a vacancy during the time step Δt , $\lambda \propto \sqrt{D_v \Delta t}$, and the mean distance between voids, $l \approx N(t)^{-1/3}$, to be a constant. However such an increase is limited by the rate of vacancy evaporation from the smallest cluster, which is equal to $Q_v(2)$.

Appendix B

In calculating the group widths it is assumed that a certain number, i_0 , of the smallest clusters are ungrouped (i.e. $\Delta x_i = 1$ at $i < i_0$), and the larger clusters are grouped, in a way that $\Delta x_i > \Delta x_{i-1}$ [3,8–10]. In contrast to [3], where the group widths are increased according to $\Delta x_i \propto x_{i-1}$, Δx_i in the present program is increased slowly, namely $\Delta x_i \propto x_{i-1}^{2/3}$. It can be shown that in this way the relative group widths in the so-called r -space, $\Delta r_i = (4\pi r_{i-1}^2/\Omega)\Delta x_i$ (r is a cluster radius related to the cluster size x by the equation $x\Omega = 4\pi r^3/3$) may be kept as a constant. Thus finally Δx_i , is given by

$$\Delta x_i = \begin{cases} 1, & i < i_0, \\ (36\pi/\Omega)^{1/3} x_{i-1}^{2/3} \Delta r, & i \geq i_0, \end{cases} \quad (\text{B.1})$$

where Δr can be varied from the input file.

Appendix C

It was found [16] that at the latest stages of coarsening of voids, when the mean void radius becomes larger and the rate of vacancy evaporation $Q(x)$ can be approximated as follows $Q(x) = D_v C_{v0} \exp(2\gamma\Omega/(r \cdot k_B T)) \approx D_v C_{v0} (1 + 2\gamma\Omega/(r \cdot k_B T))$, SDF is described by a universal function given by

$$\varphi(u, r_{cr}) = N(r_{cr}) \times \begin{cases} \frac{3^4 e}{2^{5/3}} \cdot \frac{u^2 \exp[-1/(1-2u/3)]}{(u+3)^{7/3}(3/2-u)^{11/3}}, & u < 3/2, \\ 0, & u \geq 3/2, \end{cases} \quad (\text{C.1})$$

where $N(r_{cr}) = 0.9 \cdot 3C_q / 4\pi r_{cr}^3$, $u = r/r_{cr}$, and r_{cr} is the critical cluster radius defined by the equation $J(x_{cr}) = 0$.

Acknowledgement

The work described in this paper was supported by the Research Grants Council of the Hong Kong Special Administrative Region (PolyU 99/1C) and (PolyU 5173/01E).

References

- [1] R. Kampmann, R. Wagner, in: C. Janot, W. Petry, D. Richter, T. Springer (Eds.), Atomic Transport and Defects in Metals by Neutron Scattering, Springer Proc. Phys. 10 (1986) 73.
- [2] J. Katz, H. Wiedersich, J. Chem. Phys. 55 (1971) 1414.
- [3] M. Kiritani, J. Phys. Soc. Japan 35 (1973) 95.
- [4] S.I. Golubov, Fiz. Metall. Metalloved. 52 (1981) 780; Engl. transl.: Phys. Metals Metallogr. (USSR) 52 (1981) 86.
- [5] S.I. Golubov, Yu.N. Osetsky, A. Serra, A.V. Barashev, J. Nucl. Mater. 226 (1995) 252.
- [6] M.H. Mathon, A. Barbu, F. Dunstetter, F. Maury, N. Lorenzelli, C.H. de Novion, J. Nucl. Mater. 245 (1997) 224.
- [7] S.I. Golubov, A. Serra, Yu.N. Osetsky, A.V. Barashev, J. Nucl. Mater. 277 (2000) 113.
- [8] M.R. Hyans, J. Nucl. Mater. 59 (1976) 175.
- [9] G.A. Epov, J. Nucl. Mater. 230 (1996) 84.
- [10] M. Koiwa, J. Phys. Soc. Japan 37 (1974) 1532.
- [11] S.I. Golubov, A.M. Ovcharenko, A.V. Barashev, B.N. Singh, Philos. Mag. A 81 (3) (2001) 643.
- [12] N. Myers, C++ in the real world, Dr. Dobb's J. (1997).
- [13] G. Booch, Object-Oriented Analysis and Design with Applications, 2nd Edn., Addison-Wesley, Reading, MA, 1994.
- [14] D.D. McCracken, W.S. Dorn, Numerical Methods and Fortran Programming, 2nd printing, John Wiley and Sons, 1965.
- [15] B. Stroustrup, The C++ Programming Language, 3rd edn., Addison-Wesley, Reading, MA, 1998.
- [16] I.M. Lifshitz, V.V. Slezov, Zh. Exper. Teoret. Fiz. 35 (1958) 479 (in Russian).

Input/output for GMIC++

TEST RUN INPUT

Value	Units	Name	Meaning
823.	K	t	temperature
1.205e-29	m ³	om	atomic volume
1000	mJ/m ²	ga	surface energy
1.77	eV	evf	vacancy formation energy
1.10	eV	evm	vacancy migration energy
1.00	–	cv0	pre-exp Cv0 ($C_{veq}=Cv0*\exp(-Evf/k/t)$)
1.0e-06	m ² /s	dv0	pre-exp Dv0 ($Dv=Dv0*\exp(-Evm/k/t)$)
1.0e-07	1/atom	cq	initial vacancy concentration
0.	1/m ²	kv2	sink strength
1.0le+04	s	time	terminal time for calculation
5	–	x_size	initial number of groups
10	–	x_sngl	number of groups with $dx = 1$
2.0e-11	m	dr	group width in r-space
10.5e-20	1/atom	cmax	terminal group concentration
100000	–	wrScrn	number of steps for screen output
200000	–	wrBinf	number of steps for binary output
1	–	wrTxf	number of steps for text output
1.0e-00	s	wrFrom	time of the first text-file record

TEST RUN OUTPUT

o_vac.dat

Time[s]	Cv [1/atom]
1.000e+000	8.890e-008
1.000e+001	5.129e-008
1.000e+002	5.114e-010
1.000e+003	4.259e-010
1.000e+004	1.939e-010

.....

o_sum.dat

Time[s]	N [1/atom]	S [1/atom]
1.000e+000	4.210e-009	1.085e-008
1.000e+001	2.440e-009	4.816e-008
1.000e+002	1.212e-009	9.913e-008
1.000e+003	9.860e-010	9.934e-008
1.000e+004	4.459e-010	9.984e-008

.....

o_rmv.dat

Time[s]	<x>[-]	<r>[nm]
1.000e+000	2.578e+000	1.950e-001
1.000e+001	1.974e+001	3.843e-001
1.000e+002	8.181e+001	6.174e-001
1.000e+003	1.007e+002	6.618e-001
1.000e+004	2.239e+002	8.636e-001

o_sdf.dat

Time: 1.000e+004

<x>[-]	f(<x>) [1/atom]	<r>[nm]	f(<r>) [1/nm/m3]
1.0	1.939e-010	1.422e-001	3.395e+020
2.0	1.527e-014	1.792e-001	4.243e+016
3.0	1.196e-015	2.051e-001	4.354e+015
4.0	2.785e-015	2.258e-001	1.228e+016
5.0	5.074e-015	2.432e-001	2.597e+016
6.0	7.996e-015	2.584e-001	4.622e+016
7.0	1.148e-014	2.721e-001	7.351e+016
8.0	1.544e-014	2.844e-001	1.081e+017
9.0	1.983e-014	2.958e-001	1.502e+017
10.0	2.458e-014	3.064e-001	1.997e+017
11.0	2.965e-014	3.163e-001	2.567e+017
12.0	4.155e-014	3.256e-001	3.812e+017
14.0	5.341e-014	3.428e-001	5.431e+017
16.0	6.592e-014	3.584e-001	7.327e+017
18.0	7.895e-014	3.727e-001	9.492e+017
20.5	1.003e-013	3.892e-001	1.316e+018
23.5	1.215e-013	4.074e-001	1.746e+018
26.5	1.433e-013	4.240e-001	2.230e+018
29.5	1.656e-013	4.394e-001	2.768e+018
33.0	1.972e-013	4.562e-001	3.552e+018
37.0	2.284e-013	4.739e-001	4.440e+018
41.0	2.602e-013	4.904e-001	5.415e+018
45.5	3.021e-013	5.077e-001	6.739e+018
50.5	3.437e-013	5.257e-001	8.221e+018
55.5	3.861e-013	5.425e-001	9.833e+018
61.0	4.397e-013	5.599e-001	1.193e+019
67.0	4.933e-013	5.776e-001	1.425e+019
73.5	5.588e-013	5.957e-001	1.717e+019
80.5	6.249e-013	6.141e-001	2.039e+019
88.0	7.037e-013	6.326e-001	2.437e+019
96.0	7.833e-013	6.512e-001	2.875e+019
104.5	8.764e-013	6.699e-001	3.404e+019
113.5	9.703e-013	6.886e-001	3.982e+019
123.0	1.077e-012	7.073e-001	4.665e+019
133.0	1.184e-012	7.260e-001	5.401e+019
143.5	1.301e-012	7.446e-001	6.241e+019

Time: 1.000e+004

$\langle x \rangle [-]$	$f(\langle x \rangle) [1/\text{atom}]$	$\langle r \rangle [\text{nm}]$	$f(\langle r \rangle) [1/\text{nm}/\text{m}^3]$
154.5	1.413e-012	7.631e-001	7.121e+019
166.0	1.527e-012	7.816e-001	8.072e+019
178.5	1.660e-012	8.008e-001	9.211e+019
191.5	1.776e-012	8.198e-001	1.033e+020
205.0	1.875e-012	8.386e-001	1.141e+020
219.0	1.949e-012	8.573e-001	1.240e+020
233.5	1.981e-012	8.758e-001	1.315e+020
249.0	1.967e-012	8.947e-001	1.363e+020
265.5	1.895e-012	9.141e-001	1.370e+020
282.5	1.757e-012	9.332e-001	1.324e+020
300.0	1.547e-012	9.521e-001	1.214e+020
318.5	1.278e-012	9.713e-001	1.044e+020
338.0	9.728e-013	9.907e-001	8.263e+019
358.0	6.733e-013	1.010e+000	5.942e+019
378.5	4.137e-013	1.029e+000	3.789e+019
400.0	2.153e-013	1.048e+000	2.046e+019
422.5	8.873e-014	1.067e+000	8.746e+018
446.0	2.480e-014	1.087e+000	2.534e+018
470.5	1.937e-015	1.106e+000	2.051e+017
495.5	-1.931e-015	1.125e+000	2.117e+017
521.0	-8.715e-016	1.144e+000	9.878e+016
547.5	-3.222e-017	1.163e+000	3.775e+015
575.0	8.497e-017	1.183e+000	1.028e+016
603.5	1.512e-017	1.202e+000	1.890e+015
633.0	-7.243e-018	1.221e+000	9.347e+014
663.5	-1.597e-018	1.240e+000	2.126e+014
695.0	7.350e-019	1.260e+000	1.010e+014
727.5	8.669e-020	1.279e+000	1.228e+013
761.0	-7.689e-020	1.298e+000	1.122e+013
795.5	4.738e-021	1.318e+000	7.120e+011
831.0	5.049e-021	1.337e+000	7.812e+011
867.5	-9.755e-022	1.356e+000	1.553e+011
905.0	1.136e-022	1.376e+000	1.860e+010
944.0	-6.300e-024	1.395e+000	1.061e+009